Лекция 9: типы высших родов, функторы и аппликативные функторы

Функциональное программирование на Haskell

Алексей Романов

17 октября 2026 г.

МИЭТ

Роды типов

- У значений есть типы, а у типов — ро́ды (kinds).
- Все обычные типы (`Int`, `Maybe Bool`, `(Int, Int)`, ...) имеют род `*`, или `Type` (из `Data.Kind`).

Роды типов

- У значений есть типы, а у типов — ро́ды (kinds).
- Все обычные типы (`Int`, `Maybe Bool`, `(Int, Int)`, ...) имеют род `*`, или `Type` (из `Data.Kind`).
- Конструктор типов `Maybe` принимает тип рода `*` и возвращает тип рода `*`:

```
ghci> :kind Maybe
```

Роды типов

- У значений есть типы, а у типов — ро́ды (kinds).
- Все обычные типы (`Int`, `Maybe Bool`, `(Int, Int)`, ...) имеют род `*`, или `Type` (из `Data.Kind`).
- Конструктор типов `Maybe` принимает тип рода `*` и возвращает тип рода `*`:

```
ghci> :kind Maybe
Maybe :: * -> *
```

Роды типов

- У значений есть типы, а у типов — ро́ды (kinds).
- Все обычные типы (`Int`, `Maybe Bool`, `(Int, Int)`, ...) имеют род `*`, или `Type` (из `Data.Kind`).
- Конструктор типов `Maybe` принимает тип рода `*` и возвращает тип рода `*`:

```
ghci> :kind Maybe
Maybe :: * -> *
```

- Определите род:
  - `Either` (`data Either a b = Left a | Right b`)

Роды типов

- У значений есть типы, а у типов — ро́ды (kinds).
- Все обычные типы (`Int`, `Maybe Bool`, `(Int, Int)`, ...) имеют род `*`, или `Type` (из `Data.Kind`).
- Конструктор типов `Maybe` принимает тип рода `*` и возвращает тип рода `*`:

```
ghci> :kind Maybe
Maybe :: * -> *
```

- Определите род:
  - `Either` (`data Either a b = Left a | Right b`)
  - `Shape` (`type Shape f = f ()`)

Роды типов

- У значений есть типы, а у типов — ро́ды (kinds).
- Все обычные типы (`Int`, `Maybe Bool`, `(Int, Int)`, ...) имеют род `*`, или `Type` (из `Data.Kind`).
- Конструктор типов `Maybe` принимает тип рода `*` и возвращает тип рода `*`:

```
ghci> :kind Maybe
Maybe :: * -> *
```

- Определите род:
  - `Either` (`data Either a b = Left a | Right b`)
  - `Shape` (`type Shape f = f ()`)
- В стандартном Haskell все роды строятся из `*` и `->`, в GHC всё сложнее (Kind polymorphism, Unboxed type kinds, Datatype promotion, The `Constraint` kind).

Частичное применение типов

- Конструкторы типов могут быть частично применены (как функции).
- Если `Either :: * -> * -> *`, то `Either Int`

Частичное применение типов

- Конструкторы типов могут быть частично применены (как функции).
- Если `Either :: * -> * -> *`, то `Either Int :: * -> *`. `Either a` — тоже.

Частичное применение типов

- Конструкторы типов могут быть частично применены (как функции).
- Если `Either :: * -> * -> *`, то `Either Int :: * -> *`. `Either a` — тоже.
- Типы кортежей, функций и списков можно писать в префиксном виде: `(,) a b`, `(->) a b`, `[] a`.

Частичное применение типов

- Конструкторы типов могут быть частично применены (как функции).
- Если `Either :: * -> * -> *`, то `Either Int :: * -> *`. `Either a` — тоже.
- Типы кортежей, функций и списков можно писать в префиксном виде: `(,) a b`, `(->) a b`, `[] a`.
- И применять частично: `(->) Int` читается как «функции из `Int`» и имеет род

Частичное применение типов

- Конструкторы типов могут быть частично применены (как функции).
- Если `Either :: * -> * -> *`, то `Either Int :: * -> *`. `Either a` — тоже.
- Типы кортежей, функций и списков можно писать в префиксном виде: `(,) a b`, `(->) a b`, `[] a`.
- И применять частично: `(->) Int` читается как «функции из `Int`» и имеет род `* -> *`.

Частичное применение типов

- Конструкторы типов могут быть частично применены (как функции).
- Если `Either :: * -> * -> *`, то `Either Int :: * -> *`. `Either a` — тоже.
- Типы кортежей, функций и списков можно писать в префиксном виде: `(,) a b`, `(->) a b`, `[] a`.
- И применять частично: `(->) Int` читается как «функции из `Int`» и имеет род `* -> *`.
- Синонимы типов (как `Shape` с прошлого слайда) всегда должны быть применены полностью.

Частичное применение типов

- Конструкторы типов могут быть частично применены (как функции).
- Если `Either :: * -> * -> *`, то `Either Int :: * -> *`. `Either a` — тоже.
- Типы кортежей, функций и списков можно писать в префиксном виде: `(,) a b`, `(->) a b`, `[] a`.
- И применять частично: `(->) Int` читается как «функции из `Int`» и имеет род `* -> *`.
- Синонимы типов (как `Shape` с прошлого слайда) всегда должны быть применены полностью.
- Для частичного применения нужен `newtype Shape f = Shape (f ())`.

Варианты `map`

- Сравним типы нескольких функций:
- `map :: (a -> b) -> [a] -> [b]`

Варианты `map`

- Сравним типы нескольких функций:
- `map :: (a -> b) -> [a] -> [b]`
- `mapMb :: (a -> b) -> Maybe a -> Maybe b`
  (`mapMaybe` называется другая функция)

Варианты `map`

- Сравним типы нескольких функций:
- `map :: (a -> b) -> [a] -> [b]`
- `mapMb :: (a -> b) -> Maybe a -> Maybe b`
  (`mapMaybe` называется другая функция)
- `mapMap :: (a -> b) -> Map k a -> Map k b`
  (она же `Data.Map.{Lazy/Strict}.map`)

Варианты `map`

- Сравним типы нескольких функций:
- `map :: (a -> b) -> [a] -> [b]`
- `mapMb :: (a -> b) -> Maybe a -> Maybe b`
  (`mapMaybe` называется другая функция)
- `mapMap :: (a -> b) -> Map k a -> Map k b`
  (она же `Data.Map.{Lazy/Strict}.map`)
- `mapSnd :: (a -> b) -> (c, a) -> (c, b)`

Варианты `map`

- Сравним типы нескольких функций:
- `map :: (a -> b) -> [] a -> [] b`
- `mapMb :: (a -> b) -> Maybe a -> Maybe b`
  (`mapMaybe` называется другая функция)
- `mapMap :: (a -> b) -> Map k a -> Map k b`
  (она же `Data.Map.{Lazy/Strict}.map`)
- `mapSnd :: (a -> b) -> (,) c a -> (,) c b`

Варианты `map`

- Сравним типы нескольких функций:
- `map :: (a -> b) -> [] a -> [] b`
- `mapMb :: (a -> b) -> Maybe a -> Maybe b`
  (`mapMaybe` называется другая функция)
- `mapMap :: (a -> b) -> Map k a -> Map k b`
  (она же `Data.Map.{Lazy/Strict}.map`)
- `mapSnd :: (a -> b) -> (,) c a -> (,) c b`
- Видим, что все они имеют вид

Варианты `map`

- Сравним типы нескольких функций:
- `map :: (a -> b) -> [] a -> [] b`
- `mapMb :: (a -> b) -> Maybe a -> Maybe b` (`mapMaybe` называется другая функция)
- `mapMap :: (a -> b) -> Map k a -> Map k b` (она же `Data.Map.{Lazy/Strict}.map`)
- `mapSnd :: (a -> b) -> (,) c a -> (,) c b`
- Видим, что все они имеют вид `forall a b. (a -> b) -> f a -> f b` для разных `f` (`f` не под `forall`!).

Функторы

- Можем обобщить все функции с предыдущего слайда, введя класс типов

```
class Functor f where
  fmap :: (a -> b) -> f a -> f b
  (<$) :: a        -> f b -> f a
  (<$) = fmap . const
```

- Здесь `f` — переменная типа рода

Функторы

- Можем обобщить все функции с предыдущего слайда, введя класс типов

```
class Functor f where
  fmap :: (a -> b) -> f a -> f b
  (<$) :: a        -> f b -> f a
  (<$) = fmap . const
```

- Здесь `f` — переменная типа рода `* -> *`.
- Смысл `fmap g xs`: применить функцию `g` ко всем значениям типа `a` «внутри» `xs`, не меняя структуры.
- `(<$)` — частный случай `fmap`, который иногда может быть реализован напрямую.

Функторы

- Можем обобщить все функции с предыдущего слайда, введя класс типов

```
class Functor f where
  fmap :: (a -> b) -> f a -> f b
  (<$) :: a        -> f b -> f a
  (<$) = fmap . const
```

- Здесь `f` — переменная типа рода `* -> *`.
- Смысл `fmap g xs`: применить функцию `g` ко всем значениям типа `a` «внутри» `xs`, не меняя структуры.
- `(<$)` — частный случай `fmap`, который иногда может быть реализован напрямую.
- `(<$>)` — синоним `fmap` как оператор.

Законы функторов

- Что значит «не меняя структуры»?
- Например, рассмотрим случай `fmap id`. Чему должно быть равно `fmap id xs`?

Законы функторов

- Что значит «не меняя структуры»?
- Например, рассмотрим случай `fmap id`. Чему должно быть равно `fmap id xs`?

  ```
  fmap id xs = xs
  ```

Законы функторов

- Что значит «не меняя структуры»?
- Например, рассмотрим случай `fmap id`. Чему должно быть равно `fmap id xs`?

  ```
  fmap id xs = xs
  ```
- Этот закон также формулируется как `fmap id = id`.

Законы функторов

- Что значит «не меняя структуры»?
- Например, рассмотрим случай `fmap id`. Чему должно быть равно `fmap id xs`?

  ```
  fmap id xs = xs
  ```
- Этот закон также формулируется как `fmap (id @a) = id @(f a)`.

Законы функторов

- Что значит «не меняя структуры»?
- Например, рассмотрим случай `fmap id`. Чему должно быть равно `fmap id xs`?

  ```
  fmap id xs = xs
  ```
- Этот закон также формулируется как `fmap (id @a) = id @(f a)`.
- Второй закон функторов:

  ```
  fmap (g . h) xs =
  ```

Законы функторов

- Что значит «не меняя структуры»?
- Например, рассмотрим случай `fmap id`. Чему должно быть равно `fmap id xs`?

  ```
  fmap id xs = xs
  ```
- Этот закон также формулируется как `fmap (id @a) = id @(f a)`.
- Второй закон функторов:

  ```
  fmap (g . h) xs = fmap g (fmap h xs)
  ```

Законы функторов

- Что значит «не меняя структуры»?
- Например, рассмотрим случай `fmap id`. Чему должно быть равно `fmap id xs`?

  ```
  fmap id xs = xs
  ```

- Этот закон также формулируется как `fmap (id @a) = id @(f a)`.
- Второй закон функторов:

  ```
  fmap (g . h) xs = fmap g (fmap h xs)
  fmap (g . h)    =
  ```

Законы функторов

- Что значит «не меняя структуры»?
- Например, рассмотрим случай `fmap id`. Чему должно быть равно `fmap id xs`?

  ```
  fmap id xs = xs
  ```

- Этот закон также формулируется как `fmap (id @a) = id @(f a)`.
- Второй закон функторов:

  ```
  fmap (g . h) xs = fmap g (fmap h xs)
  fmap (g . h)    = fmap g . fmap h
  ```

Примеры функторов

- Очень многие типы (рода `* -> *`) являются функторами. Например:

```
instance Functor Maybe where
  -- fmap :: (a -> b) -> Maybe a -> Maybe b
  fmap g (Just x) =
```

Примеры функторов

- Очень многие типы (рода `* -> *`) являются функторами. Например:

```
instance Functor Maybe where
  -- fmap :: (a -> b) -> Maybe a -> Maybe b
  fmap g (Just x) = Just (g x)
  fmap _ Nothing  =
```

Примеры функторов

- Очень многие типы (рода `* -> *`) являются функторами. Например:

```
instance Functor Maybe where
  -- fmap :: (a -> b) -> Maybe a -> Maybe b
  fmap g (Just x) = Just (g x)
  fmap _ Nothing  = Nothing
```

Примеры функторов

- Очень многие типы (рода `* -> *`) являются функторами. Например:

```
instance Functor Maybe where
  -- fmap :: (a -> b) -> Maybe a -> Maybe b
  fmap g (Just x) = Just (g x)
  fmap _ Nothing  = Nothing
```

- `instance Functor [] where`

```
  fmap =
```

Примеры функторов

- Очень многие типы (рода `* -> *`) являются функторами. Например:

```
instance Functor Maybe where
  -- fmap :: (a -> b) -> Maybe a -> Maybe b
  fmap g (Just x) = Just (g x)
  fmap _ Nothing  = Nothing
```

-
```
instance Functor [] where
  fmap = map
```

- Не следует определять

```
instance Functor [] where
  fmap g xs = []
```

Примеры функторов

- Очень многие типы (рода `* -> *`) являются функторами. Например:

```
instance Functor Maybe where
  -- fmap :: (a -> b) -> Maybe a -> Maybe b
  fmap g (Just x) = Just (g x)
  fmap _ Nothing  = Nothing
```

-
```
instance Functor [] where
  fmap = map
```

- Не следует определять

```
instance Functor [] where
  fmap g xs = []
```

  Типы сходятся (проверьте!), но законы нарушены.

Ещё примеры функторов

- Частично применённые кортежи:

```haskell
instance Functor ((,) c) where
  -- fmap :: (a -> b) -> (c, a) -> (c, b)
  fmap g (z, x) =
```

Ещё примеры функторов

- Частично применённые кортежи:

```haskell
instance Functor ((,) c) where
  -- fmap :: (a -> b) -> (c, a) -> (c, b)
  fmap g (z, x) = (z, g x)
```

Ещё примеры функторов

- Частично применённые кортежи:

```
instance Functor ((,) c) where
  -- fmap :: (a -> b) -> (c, a) -> (c, b)
  fmap g (z, x) = (z, g x)
```

-
```
data Pair a = Pair a a
instance Functor Pair where
  -- fmap :: (a -> b) -> Pair a -> Pair b
  fmap g (Pair x y) =
```

Ещё примеры функторов

- Частично применённые кортежи:

```
instance Functor ((,) c) where
  -- fmap :: (a -> b) -> (c, a) -> (c, b)
  fmap g (z, x) = (z, g x)
```

-
```
data Pair a = Pair a a
instance Functor Pair where
  -- fmap :: (a -> b) -> Pair a -> Pair b
  fmap g (Pair x y) = Pair (g x) (g y)
```

Ещё примеры функторов

- Частично применённые кортежи:

```
instance Functor ((,) c) where
  -- fmap :: (a -> b) -> (c, a) -> (c, b)
  fmap g (z, x) = (z, g x)
```

-
```
data Pair a = Pair a a
instance Functor Pair where
  -- fmap :: (a -> b) -> Pair a -> Pair b
  fmap g (Pair x y) = Pair (g x) (g y)
```

- Дома (начните с уточнения типа `fmap`):

```
instance Functor (Either c) where ...
instance Functor ((->) c) where ...
```

… и нефункторов

- Не все типы можно сделать функторами!
  Рассмотрим

```haskell
newtype Pred a = Pred (a -> Bool)
instance Functor Pred where
  -- fmap :: (a -> b) -> Pred a -> Pred b
  fmap g (Pred p) =
```

… и нефункторов

- Не все типы можно сделать функторами!
  Рассмотрим

```haskell
newtype Pred a = Pred (a -> Bool)
instance Functor Pred where
  -- fmap :: (a -> b) -> Pred a -> Pred b
  fmap g (Pred p) = Pred (\x -> ???)
```

… и нефункторов

- Не все типы можно сделать функторами! Рассмотрим

```
newtype Pred a = Pred (a -> Bool)
instance Functor Pred where
  -- fmap :: (a -> b) -> Pred a -> Pred b
  fmap g (Pred p) = Pred (\x -> ???)
```

`g :: a -> b` не к чему применить: у нас нет ничего типа `a`!

Функторы: полярность параметра

- Вхождение `a` *положительно*, если оно слева от чётного числа `->` (в том числе 0), и *отрицательно*, если от нечётного.
- Для `Functor` все вхождения `a` должны быть положительными.
- Во вложенных конструкторах типов тоже должны быть подходящие операции отображения (правила deriving).

Функторы: полярность параметра

- Вхождение a *положительно*, если оно слева от чётного числа `->` (в том числе 0), и *отрицательно*, если от нечётного.
- Для `Functor` все вхождения a должны быть положительными.
- Во вложенных конструкторах типов тоже должны быть подходящие операции отображения (правила deriving).
- По этому правилу, функтор ли ниже?

```
newtype Tricky a = T ((a -> Int) -> a)
```

Доказательство законов для экземпляра `Functor`

- Возьмём для примера `instance Functor Pair`. Нужно доказать, что для него:
- $\forall$ `a :: *, pair :: Pair a`

  ```
  fmap id pair = pair
  ```

Доказательство законов для экземпляра `Functor`

- Возьмём для примера `instance Functor Pair`. Нужно доказать, что для него:
- `∀ a :: *, pair :: Pair a`

  `fmap id pair = pair`
- Пусть `pair = Pair x y`, где `x, y ::`

Доказательство законов для экземпляра `Functor`

- Возьмём для примера `instance Functor Pair`. Нужно доказать, что для него:
- $\forall$ `a :: *`, `pair :: Pair a`

  ```
  fmap id pair = pair
  ```
- Пусть `pair = Pair x y`, где `x, y :: a`

  ```
  fmap id pair =
  fmap id (Pair x y) =
  ```

Доказательство законов для экземпляра `Functor`

- Возьмём для примера `instance Functor Pair`. Нужно доказать, что для него:
- $\forall$ `a :: *`, `pair :: Pair a`

  ```
  fmap id pair = pair
  ```
- Пусть `pair = Pair x y`, где `x, y :: a`

  ```
  fmap id pair =
  fmap id (Pair x y) =  -- определение fmap
  ```

Доказательство законов для экземпляра `Functor`

- Возьмём для примера `instance Functor Pair`. Нужно доказать, что для него:
- $\forall$ `a :: *`, `pair :: Pair a`

  ```
  fmap id pair = pair
  ```
- Пусть `pair = Pair x y`, где `x, y :: a`

  ```
  fmap id pair =
  fmap id (Pair x y) =  -- определение fmap
  Pair (id x) (id y) =
  ```

Доказательство законов для экземпляра `Functor`

- Возьмём для примера `instance Functor Pair`.
  Нужно доказать, что для него:
- $\forall$ `a :: *`, `pair :: Pair a`

  ```
  fmap id pair = pair
  ```
- Пусть `pair = Pair x y`, где `x, y :: a`

  ```
  fmap id pair =
  fmap id (Pair x y) =  -- определение fmap
  Pair (id x) (id y) =  -- определение id
  ```

Доказательство законов для экземпляра `Functor`

- Возьмём для примера `instance Functor Pair`. Нужно доказать, что для него:
- $\forall$ `a :: *`, `pair :: Pair a`

  ```
  fmap id pair = pair
  ```
- Пусть `pair = Pair x y`, где `x, y :: a`

  ```
  fmap id pair =
  fmap id (Pair x y) =  -- определение fmap
  Pair (id x) (id y) =  -- определение id
  Pair x y =
  pair
  ```

Доказательство законов для экземпляра `Functor`

- Возьмём для примера `instance Functor Pair`. Нужно доказать, что для него:
- $\forall$ `a :: *, pair :: Pair a`

  ```
  fmap id pair = pair
  ```
- Пусть `pair = Pair x y`, где `x, y :: a`

  ```
  fmap id pair =
  fmap id (Pair x y) =  -- определение fmap
  Pair (id x) (id y) =  -- определение id
  Pair x y =
  pair
  ```
- Докажите самостоятельно: $\forall$ `a, b, c :: *, pair :: Pair a, h :: a -> b, g :: b -> c`

  ```
  fmap (g . h) pair = fmap g (fmap h pair)
  ```

Использование функторов

- Предскажите результаты:

```
(+ 3) <$> Just 1
(+ 3) <$> Nothing
(+ 3) <$> [1, 2, 3]
```

Использование функторов

- Предскажите результаты:

```
(+ 3) <$> Just 1
(+ 3) <$> Nothing
(+ 3) <$> [1, 2, 3]
```

- ```
  void :: Functor f => f a -> f ()
  void x = ???

  void [1, 2]
  ```
  ```

Чего не могут функторы?

- Удобно считать, что `fmap` берёт функцию `a -> b` и «поднимает» её в тип `f a -> f b`.
- Можно ли с её помощью поднять функцию двух аргументов? Т.е. можно ли реализовать

```
fmap2 :: Functor f => (a -> b -> c) ->
  f a -> f b -> f c
fmap2 g fx fy = ???
```

Чего не могут функторы?

- Удобно считать, что `fmap` берёт функцию `a -> b` и «поднимает» её в тип `f a -> f b`.
- Можно ли с её помощью поднять функцию двух аргументов? Т.е. можно ли реализовать

```
fmap2 :: Functor f => (a -> b -> c) ->
  f a -> f b -> f c
fmap2 g fx fy = ???
```

- Нет! Единственное, с чего мы можем начать:

Чего не могут функторы?

- Удобно считать, что `fmap` берёт функцию `a -> b` и «поднимает» её в тип `f a -> f b`.
- Можно ли с её помощью поднять функцию двух аргументов? Т.е. можно ли реализовать

```
fmap2 :: Functor f => (a -> b -> c) ->
  f a -> f b -> f c
fmap2 g fx fy = ???
```

- Нет! Единственное, с чего мы можем начать: `fmap g`. Она имеет тип

Чего не могут функторы?

- Удобно считать, что `fmap` берёт функцию `a -> b` и «поднимает» её в тип `f a -> f b`.
- Можно ли с её помощью поднять функцию двух аргументов? Т.е. можно ли реализовать

```
fmap2 :: Functor f => (a -> b -> c) ->
  f a -> f b -> f c
fmap2 g fx fy = ???
```

- Нет! Единственное, с чего мы можем начать: `fmap g`. Она имеет тип `f a -> f (b -> c)`.
- Применив к `fx` (тип подходит), получим `fmap g fx :: f (b -> c)`.

Чего не могут функторы?

- Удобно считать, что `fmap` берёт функцию `a -> b` и «поднимает» её в тип `f a -> f b`.
- Можно ли с её помощью поднять функцию двух аргументов? Т.е. можно ли реализовать

```
fmap2 :: Functor f => (a -> b -> c) ->
  f a -> f b -> f c
fmap2 g fx fy = ???
```

- Нет! Единственное, с чего мы можем начать: `fmap g`. Она имеет тип `f a -> f (b -> c)`.
- Применив к `fx` (тип подходит), получим `fmap g fx :: f (b -> c)`.
- Но `f (b -> c)` и `f b` скомбинировать уже не получится.

Аппликативные функторы

- К `Functor` добавим поднятие значения и применение функции в контексте:

```
class Functor f => Applicative f where
  pure  :: a -> f a
  infixl 4 <*>, *>, <*
  (<*>) :: f (a -> b) -> f a -> f b
  liftA2 :: (a -> b -> c) ->
    f a -> f b -> f c
  (*>) :: f a -> f b -> f b
  fx *> fy = (id <$ fx) <*> fy
  (<*) :: f a -> f b -> f a
  fx <* fy = const <$> fx <*> fy
```

- `(*>)` и `(<*)` комбинируют контексты, как `(<*>)`: `(*>)` сохраняет значения справа, игнорируя значения слева; `(<*)` — наоборот.

Аппликативные функторы: определения по умолчанию

- `(<*>)` и `liftA2` имеют взаимные определения по умолчанию.
- Упражнение: выразите каждый метод через другой.

```
liftA2 g fx fy = ???

fs <*> xs = ???
```

Законы аппликативных функторов

- У `Applicative` 5 законов:
- ```
  -- тождество (identity)
  pure id <*> v = v
  -- гомоморфизм (homomorphism)
  pure g <*> pure x = pure (g x)
  -- перестановка (interchange)
  u <*> pure y = pure ($ y) <*> u
  -- композиция (composition)
  u <*> (v <*> w) = pure (.) <*> u <*> v <*> w
  -- согласованность с fmap
  fmap g x = pure g <*> x
  ```
  ```

Моноидальные функторы

- Эти законы не очень удобны и их трудно запомнить. Но мы можем использовать другой класс:
- ```
  class Functor f => Monoidal f where
    unit :: f ()
    pair :: f a -> f b -> f (a,b)
  ```
- Он взаимовыразим с аппликативными функторами (т.е. `pure` и `(<*>)` выражаются через `unit` и `pair` и наоборот).
- Законы ($\simeq$: с точностью до изоморфизма):
  ```
  pair unit v ≃ v
  pair u unit ≃ u
  pair u (pair v w) ≃ pair (pair u v) w
  ```
- Упражнение: записать их точные версии.
  ```

Примеры аппликативных функторов

- Многие функторы являются аппликативными:

```
instance Applicative Maybe where
  -- pure :: a -> Maybe a
  pure =
```

Примеры аппликативных функторов

- Многие функторы являются аппликативными:

```
instance Applicative Maybe where
  -- pure :: a -> Maybe a
  pure = Just
  -- (<*>) :: Maybe (a -> b) -> Maybe a ->
  --           Maybe b
  Just g <*> Just x  =
```

Примеры аппликативных функторов

- Многие функторы являются аппликативными:

```
instance Applicative Maybe where
  -- pure :: a -> Maybe a
  pure = Just
  -- (<*>) :: Maybe (a -> b) -> Maybe a ->
  --           Maybe b
  Just g <*> Just x  = Just (g x)
  _      <*> _       =
```

Примеры аппликативных функторов

- Многие функторы являются аппликативными:

```haskell
instance Applicative Maybe where
  -- pure :: a -> Maybe a
  pure = Just
  -- (<*>) :: Maybe (a -> b) -> Maybe a ->
  --           Maybe b
  Just g <*> Just x  = Just (g x)
  _      <*> _       = Nothing
```

Примеры аппликативных функторов

- Многие функторы являются аппликативными:

```
instance Applicative Maybe where
  -- pure :: a -> Maybe a
  pure = Just
  -- (<*>) :: Maybe (a -> b) -> Maybe a ->
  --           Maybe b
  Just g <*> Just x  = Just (g x)
  _      <*> _       = Nothing
```

- Для списков есть 2 варианта. Стандартный:

```
instance Applicative [] where
  -- pure :: a -> [a]
  pure x =
```

Примеры аппликативных функторов

- Многие функторы являются аппликативными:

```
instance Applicative Maybe where
  -- pure :: a -> Maybe a
  pure = Just
  -- (<*>) :: Maybe (a -> b) -> Maybe a ->
  --           Maybe b
  Just g <*> Just x  = Just (g x)
  _      <*> _       = Nothing
```

- Для списков есть 2 варианта. Стандартный:

```
instance Applicative [] where
  -- pure :: a -> [a]
  pure x = [x]
  -- (<*>) :: [a -> b] -> [a] -> [b]
  gs <*> xs =
```

Примеры аппликативных функторов

- Многие функторы являются аппликативными:

```
instance Applicative Maybe where
  -- pure :: a -> Maybe a
  pure = Just
  -- (<*>) :: Maybe (a -> b) -> Maybe a ->
  --          Maybe b
  Just g <*> Just x  = Just (g x)
  _      <*> _       = Nothing
```

- Для списков есть 2 варианта. Стандартный:

```
instance Applicative [] where
  -- pure :: a -> [a]
  pure x = [x]
  -- (<*>) :: [a -> b] -> [a] -> [b]
  gs <*> xs = [g x | g <- gs, x <- xs]
```

Примеры аппликативных функторов

- Второй вариант для списков — `ZipList` (определён в `Control.Applicative`).

```
newtype ZipList a = ZipList { getZipList :: [a] }
  deriving Functor
instance Applicative ZipList where
  (ZipList fs) <*> (ZipList xs) =
    ZipList (zipWith ($) fs xs)
  pure x = ???
```

Примеры аппликативных функторов

- Второй вариант для списков — `ZipList` (определён в `Control.Applicative`).

```
newtype ZipList a = ZipList { getZipList :: [a] }
  deriving Functor
instance Applicative ZipList where
  (ZipList fs) <*> (ZipList xs) =
    ZipList (zipWith ($) fs xs)
  pure x = ???
```

- Подсказка:
  - `zipWith` обрывается по более короткому списку.
  - Реализация должна удовлетворять закону

    ```
    pure id <*> v = v
    ```

Примеры аппликативных функторов

- Второй вариант для списков — `ZipList` (определён в `Control.Applicative`).

```
newtype ZipList a = ZipList { getZipList :: [a] }
  deriving Functor
instance Applicative ZipList where
  (ZipList fs) <*> (ZipList xs) =
    ZipList (zipWith ($) fs xs)
  pure x = ???
```

- Подсказка:
  - `zipWith` обрывается по более короткому списку.
  - Реализация должна удовлетворять закону

    ```
    pure id <*> v = v
    ```

- Ответ:

```
pure x = ZipList (repeat x)
```

Собираем значение с помощью Applicative

- Конструктор тоже можно применить к значениям в контексте:

```
import Text.Read (readMaybe)

data Point = Point Int Int deriving Show

parsePoint :: String -> String -> Maybe Point
parsePoint sx sy =
  Point <$> readMaybe sx <*> readMaybe sy
```

Собираем значение с помощью Applicative

- Конструктор тоже можно применить к значениям в контексте:

```
import Text.Read (readMaybe)

data Point = Point Int Int deriving Show

parsePoint :: String -> String -> Maybe Point
parsePoint sx sy =
  Point <$> readMaybe sx <*> readMaybe sy
```

- Если обе координаты удалось прочитать, получим точку:

```
ghci> parsePoint "2" "3"
Just (Point 2 3)
ghci> parsePoint "2" "x"
Nothing
```

Аппликативный функтор с ограничением

- ```
  instance Applicative ((,) c) where
    -- pure :: a -> (c, a)
    pure = ???
  ```
  ```

Аппликативный функтор с ограничением

- ```
  instance Applicative ((,) c) where
    -- pure :: a -> (c, a)
    pure = ???
  ```

  Невозможно определить без ограничений на `c`.
  ```

Аппликативный функтор с ограничением

- ```
  instance Applicative ((,) c) where
    -- pure :: a -> (c, a)
    pure = ???
  ```

  Невозможно определить без ограничений на `c`.
- А какие ограничения нужны?
  ```

Аппликативный функтор с ограничением

- ```
  instance Applicative ((,) c) where
    -- pure :: a -> (c, a)
    pure = ???
  ```

  Невозможно определить без ограничений на `c`.

- А какие ограничения нужны?

  ```
  instance Monoid c => Applicative ((,) c) where
    pure x = (mempty, x)
    (c1, g) <*> (c2, x) =
  ```
  ```

Аппликативный функтор с ограничением

- ```
  instance Applicative ((,) c) where
    -- pure :: a -> (c, a)
    pure = ???
  ```

  Невозможно определить без ограничений на `c`.

- А какие ограничения нужны?

  ```
  instance Monoid c => Applicative ((,) c) where
    pure x = (mempty, x)
    (c1, g) <*> (c2, x) = (c1 <> c2, g x)
  ```
  ```

Что можно сделать с аппликативными функторами

- Предскажите результаты:

```
pure (+) <*> [2, 3, 4] <*> pure 4
[(+1), (*2)] <*> [2, 3]
[1, 2] *> [3, 4, 5]
[1, 2] <* [3, 4, 5]
ZipList [1, 2] *> ZipList [3, 4, 5]
```

Что можно сделать с аппликативными функторами

- Предскажите результаты:

```
pure (+) <*> [2, 3, 4] <*> pure 4
[(+1), (*2)] <*> [2, 3]
[1, 2] *> [3, 4, 5]
[1, 2] <* [3, 4, 5]
ZipList [1, 2] *> ZipList [3, 4, 5]
```

- Реализуйте

```
when :: Applicative f => Bool -> f () -> f ()
sequenceAL :: Applicative f => [f a] -> f [a]
```

- У второй есть обобщённая версия `sequenceA` из класса `Traversable` (ниже).

… и что нельзя

- Структура результата аппликативного вычисления зависит только от структуры аргументов, а не от значений внутри них.
- Т.е. мы не можем определить функцию

```
ifA :: Applicative f => f Bool ->
  f a -> f a -> f a
```

  так, чтобы выполнялось

```
ifA [True]  [1, 2] [3] == [1, 2]
ifA [False] [1, 2] [3] == [3]
```

  (в случае списков структура — число элементов).

… и что нельзя

- Структура результата аппликативного вычисления зависит только от структуры аргументов, а не от значений внутри них.
- Т.е. мы не можем определить функцию

```
ifA :: Applicative f => f Bool ->
  f a -> f a -> f a
```

  так, чтобы выполнялось

```
ifA [True]  [1, 2] [3] == [1, 2]
ifA [False] [1, 2] [3] == [3]
```

  (в случае списков структура — число элементов).
- Попробуйте написать какие-нибудь варианты `ifA` по типу и убедиться в этом.

… и что нельзя

- Структура результата аппликативного вычисления зависит только от структуры аргументов, а не от значений внутри них.
- Т.е. мы не можем определить функцию

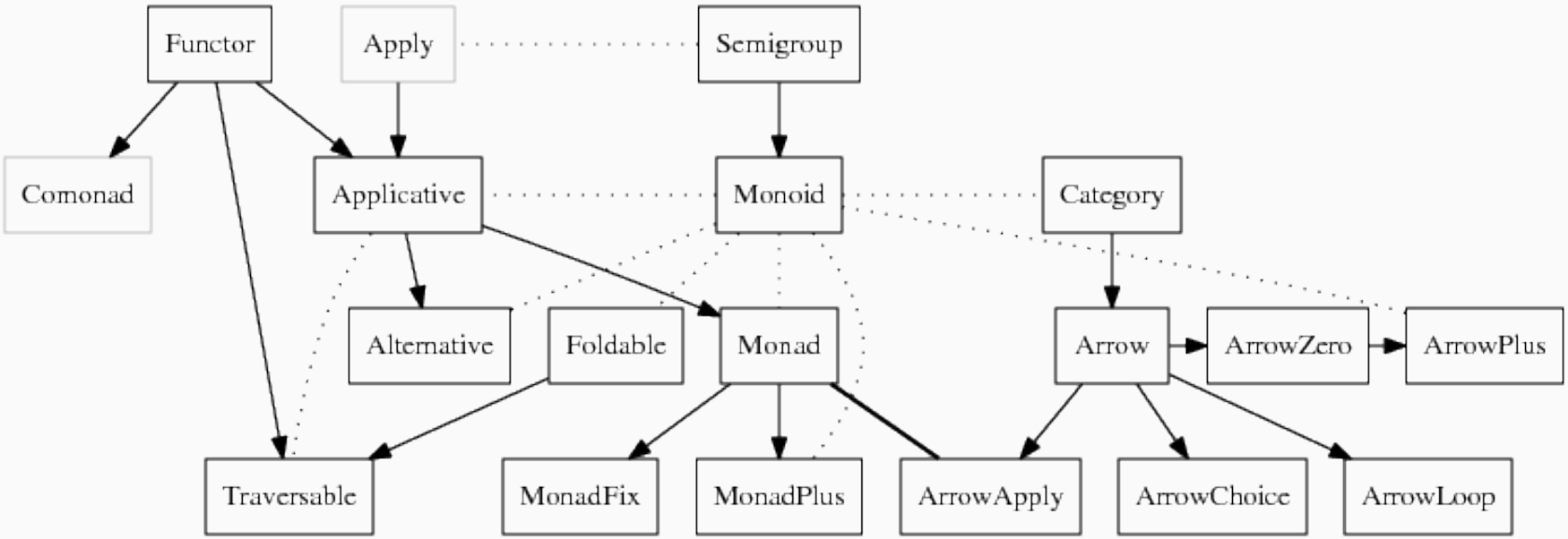
```
ifA :: Applicative f => f Bool ->
  f a -> f a -> f a
```

  так, чтобы выполнялось

```
ifA [True]  [1, 2] [3] == [1, 2]
ifA [False] [1, 2] [3] == [3]
```

  (в случае списков структура — число элементов).
- Попробуйте написать какие-нибудь варианты `ifA` по типу и убедиться в этом.
- Эту проблему решают монады, о которых будет следующая лекция.

Диаграмма классов типов высших родов



Foldable

- В лекции 6 мы видели свёртки списков, `Maybe` и деревьев. Обобщение — класс

```
class Foldable t where
  foldr   :: (a -> b -> b) -> b -> t a -> b
  foldMap :: Monoid m => (a -> m) -> t a -> m
  -- достаточно определить одну из двух
  toList, length, null, elem, sum, foldl', ...
```

- Поэтому у `length`, `null`, `sum`, `elem` в `Prelude` на самом деле более общие типы (лекция 6).

Foldable

- В лекции 6 мы видели свёртки списков, `Maybe` и деревьев. Обобщение — класс

```
class Foldable t where
  foldr   :: (a -> b -> b) -> b -> t a -> b
  foldMap :: Monoid m => (a -> m) -> t a -> m
  -- достаточно определить одну из двух
  toList, length, null, elem, sum, foldl', ...
```

- Поэтому у `length`, `null`, `sum`, `elem` в `Prelude` на самом деле более общие типы (лекция 6).
- `foldMap g = foldr (\x acc -> g x <> acc) mempty`.

`Foldable`: моноиды и ловушки

- Сумма через моноид `Sum` из лекции 4:

```
import Data.Foldable (foldMap')
import Data.Monoid (Sum(..))
sum :: (Foldable t, Num a) => t a -> a
sum = getSum . foldMap' Sum
```

  `foldMap'` — строгая свёртка, как `foldl'` в лекции 8. Так и определена `sum` в `base`.

`Foldable`: моноиды и ловушки

- Сумма через моноид `Sum` из лекции 4:

```
import Data.Foldable (foldMap')
import Data.Monoid (Sum(..))
sum :: (Foldable t, Num a) => t a -> a
sum = getSum . foldMap' Sum
```

  `foldMap'` — строгая свёртка, как `foldl'` в лекции 8. Так и определена `sum` в `base`.

- Ловушка: `length (1, 2) == 1` — экземпляр `Foldable ((,) c)` «видит» только второй компонент пары.

Traversable

- `sequenceAL` обобщается со списков на структуры класса `Traversable`:

```
class (Functor t, Foldable t) => Traversable t where
  traverse  :: Applicative f => (a -> f b) -> t a -> f (t b)
  sequenceA :: Applicative f => t (f a) -> f (t a)
  -- sequenceA =
```

Traversable

- `sequenceAL` обобщается со списков на структуры класса `Traversable`:

```
class (Functor t, Foldable t) => Traversable t where
  traverse  :: Applicative f => (a -> f b) -> t a -> f (t b)
  sequenceA :: Applicative f => t (f a) -> f (t a)
  -- sequenceA =  traverse id
  -- traverse g =
```

Traversable

- `sequenceAL` обобщается со списков на структуры класса `Traversable`:

```
class (Functor t, Foldable t) => Traversable t where
  traverse  :: Applicative f => (a -> f b) -> t a -> f (t b)
  sequenceA :: Applicative f => t (f a) -> f (t a)
  -- sequenceA =  traverse id
  -- traverse g =  sequenceA . fmap g
```

Traversable

- `sequenceAL` обобщается со списков на структуры класса `Traversable`:

```
class (Functor t, Foldable t) => Traversable t where
  traverse  :: Applicative f => (a -> f b) -> t a -> f (t b)
  sequenceA :: Applicative f => t (f a) -> f (t a)
  -- sequenceA =  traverse id
  -- traverse g =  sequenceA . fmap g
```

- `traverse` — «`map` с эффектами»:

```
ghci> import Text.Read (readMaybe)
ghci> traverse readMaybe ["1", "2"] :: Maybe [Int]
```

Traversable

- `sequenceAL` обобщается со списков на структуры класса `Traversable`:

```
class (Functor t, Foldable t) => Traversable t where
  traverse  :: Applicative f => (a -> f b) -> t a -> f (t b)
  sequenceA :: Applicative f => t (f a) -> f (t a)
  -- sequenceA =  traverse id
  -- traverse g =  sequenceA . fmap g
```

- `traverse` — «`map` с эффектами»:

```
ghci> import Text.Read (readMaybe)
ghci> traverse readMaybe ["1", "2"] :: Maybe [Int]
Just [1,2]
ghci> traverse readMaybe ["1", "x"] :: Maybe [Int]
```

Traversable

- `sequenceAL` обобщается со списков на структуры класса `Traversable`:

```
class (Functor t, Foldable t) => Traversable t where
  traverse  :: Applicative f => (a -> f b) -> t a -> f (t b)
  sequenceA :: Applicative f => t (f a) -> f (t a)
  -- sequenceA =  traverse id
  -- traverse g =  sequenceA . fmap g
```

- `traverse` — «`map` с эффектами»:

```
ghci> import Text.Read (readMaybe)
ghci> traverse readMaybe ["1", "2"] :: Maybe [Int]
Just [1,2]
ghci> traverse readMaybe ["1", "x"] :: Maybe [Int]
Nothing
```

Traversable

- `sequenceAL` обобщается со списков на структуры класса `Traversable`:

```
class (Functor t, Foldable t) => Traversable t where
  traverse  :: Applicative f => (a -> f b) -> t a -> f (t b)
  sequenceA :: Applicative f => t (f a) -> f (t a)
  -- sequenceA =  traverse id
  -- traverse g =  sequenceA . fmap g
```

- `traverse` — «`map` с эффектами»:

```
ghci> import Text.Read (readMaybe)
ghci> traverse readMaybe ["1", "2"] :: Maybe [Int]
Just [1,2]
ghci> traverse readMaybe ["1", "x"] :: Maybe [Int]
Nothing
```

- `deriving (Functor, Foldable, Traversable)` работает для большинства типов. Законы — в `Data.Traversable`.

Выполняем действия без сбора результатов

- Суффикс `_` обычно означает: результаты отбрасываются, эффекты сохраняются.
- Для `sequenceA` это `sequenceA_`:

```
import Data.Foldable (sequenceA_)

sequenceA_ :: (Foldable t, Applicative f) =>
  t (f a) -> f ()
```

Выполняем действия без сбора результатов

- Суффикс _ обычно означает: результаты отбрасываются, эффекты сохраняются.
- Для `sequenceA` это `sequenceA_`:

```
import Data.Foldable (sequenceA_)

sequenceA_ :: (Foldable t, Applicative f) =>
  t (f a) -> f ()
```

- Значения отбрасываются, но отказ сохраняется:

```
ghci> sequenceA [Just 1, Just 2]
Just [1,2]
ghci> sequenceA_ [Just 1, Just 2]
Just ()
ghci> sequenceA_ [Just 1, Nothing]
Nothing
```

Дополнительное чтение

- Typeclassopedia: справочник и диаграмма классов типов.
- Good examples of Not a Functor/Applicative/Monad?: контрпримеры.
- Functors, Applicatives, And Monads In Pictures: наглядное введение.
- Applicative Functors в Wikibook: примеры и упражнения.
- Learn You a Haskell, гл. 11: учебный разбор (некоторые определения устарели).
- C. McBride, R. Paterson, Applicative Programming with Effects: исходная статья про `Applicative`.
- J. Gibbons, B. Oliveira, The Essence of the Iterator Pattern: глубже о `Traversable`.